

Guía del programador

Construcción y organización interna del sistema **VorTrack**: tecnologías utilizadas, arquitectura por capas y ficha detallada de cada módulo (archivo) con sus importaciones y funciones principales.

Proyecto: VorTrack — trazabilidad PET → filamento → impresión · **Institución:** Instituto San Carlos de La Salle

Lenguaje: Python 3.11 · **Base de datos:** Microsoft SQL Server · **Autor:** Miguel Ruiz Ramirez · **Año:** 2026

Repositorio público: github.com/VorTrack-git/Vortrack.exe · **Versión de la guía:** septiembre 2026

Contenido

- | | |
|--|---|
| 1 Diagrama de flujo | 8 Raíz y composición |
| 2 Arquitectura general | 9 Capa de datos (acceso a la base de datos) |
| 3 Tecnologías utilizadas | 10 Servicios (lógica de negocio) |
| 4 Estructura de carpetas | 11 Interfaz de usuario |
| 5 Repositorio en GitHub | 12 Pruebas |
| 6 Carga en paralelo y experiencia de uso | 13 Scripts y utilidades |
| 7 Informe PDF | |

1 Diagrama de flujo

Recorrido general del sistema: desde el inicio de sesión (con validación de credenciales) hasta el registro de la operación y su guardado en la base de datos. Los rombos son decisiones y las flechas punteadas de retorno son bucles («reintentar» / «corregir»).

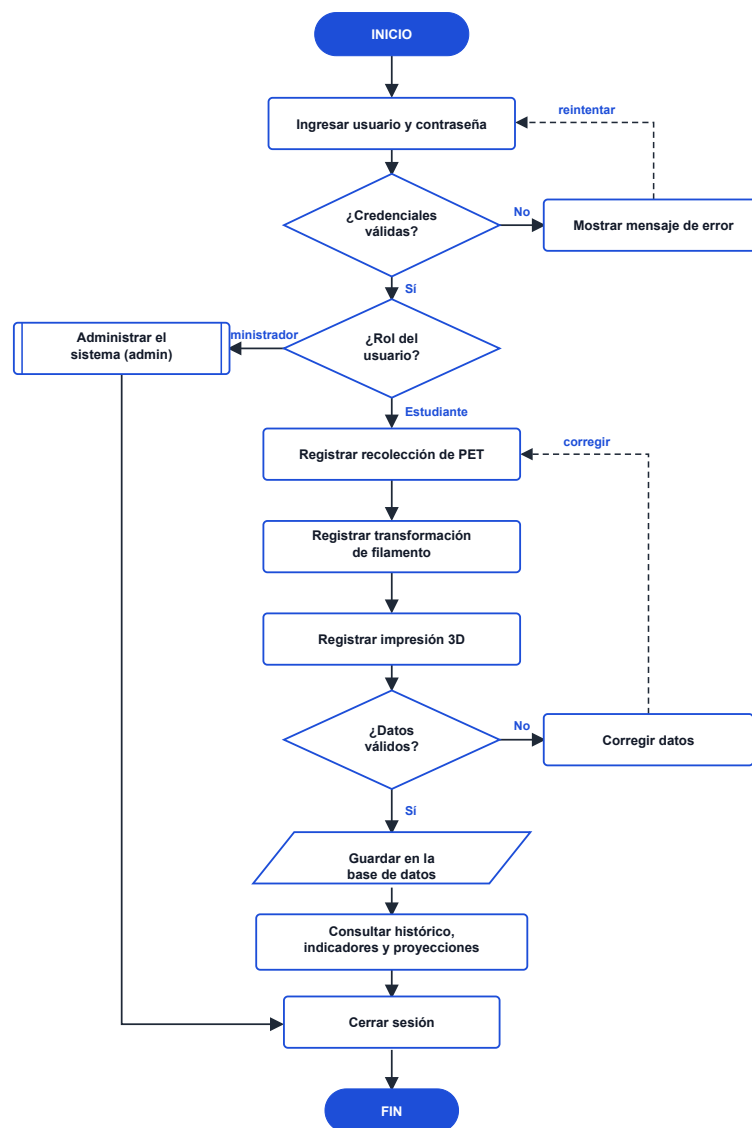


Fig. 1 – Flujo del sistema VorTrack.

2 Arquitectura general

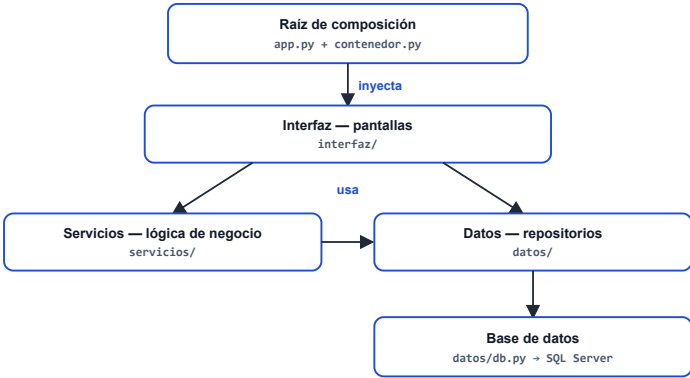


Fig. 2 – Capas del sistema y dirección de las dependencias.

El proyecto está organizado en **capas** con una sola dirección de dependencia (de arriba hacia abajo). Esto hace que cada parte tenga una única responsabilidad y sea fácil de mantener:

- **Interfaz** (carpeta `interfaz/`): las pantallas que ve el usuario. No sabe de SQL; pide y muestra datos a través de objetos que recibe.
- **Servicios** (`servicios/`): la lógica de negocio (autenticación, proyecciones, validaciones).
- **Datos** (`datos/`): los repositorios que convierten operaciones en consultas SQL, y la conexión.
- **Infraestructura**: `datos/db.py` es lo único que conoce el detalle de SQL Server.

La **inyección de dependencias** es la idea clave: las pantallas no crean sus repositorios ni servicios, sino que los *reciben* ya construidos. El único lugar que arma todo es la **raíz de composición** (`app.py + contenedor.py`). Así se puede cambiar una implementación sin tocar las pantallas.

Fachadas. `datos/repositorio.py` y `servicios/auth.py` son *fachadas*: agrupan bajo una interfaz sencilla el trabajo que por dentro reparten varios módulos. La interfaz habla con la fachada y no necesita conocer el detalle.

3 Tecnologías utilizadas

VorTrack es una aplicación de **escritorio** escrita en Python. La interfaz usa tkinter (y, si está instalada, customtkinter para un aspecto moderno) y los datos se guardan en SQL Server. Estas son las herramientas y para qué se usan:

HERRAMIENTA	TIPO	PARA QUÉ SE USA EN VORTACK
Python 3.11	Lenguaje	Lenguaje de programación base de todo el proyecto.
tkinter	Estándar	Framework de interfaz gráfica: ventanas, formularios y tablas.
customtkinter	Externa	Widgets modernos sobre tkinter (tema oscuro, esquinas redondeadas). La app funciona con o sin ella.
pyodbc	Externa	Conecta la aplicación con SQL Server mediante el controlador ODBC.
Pillow (PIL)	Externa	Carga y redimensiona el logo y los íconos.
tkcalendar	Externa	Selector de fecha (DateEntry) en los formularios.

HERRAMIENTA	TIPO	PARA QUÉ SE USA EN VORTTRACK
<code>concurrent.futures</code>	Estándar	ThreadPoolExecutor: ejecuta varias consultas a la base de datos al mismo tiempo para que las pantallas carguen más rápido.
Microsoft Edge (headless)	Sistema	Convierte la maqueta HTML del informe a PDF (--print-to-pdf). Viene instalado con Windows; si falta, se usa Google Chrome.
<code>hashlib · configparser · json · datetime · os · sys · typing · getpass</code>	Estándar	Cifrado de contraseñas, lectura del .ini, archivos JSON, fechas, rutas, tipos y contraseña por consola.
<code>subprocess · tempfile · shutil · ctypes · base64 · html · pathlib · collections · math · time</code>	Estándar	Ejecutar el navegador, archivos temporales, ubicar Descargas (API de Windows), armar el HTML del informe, agrupar estadísticas y animaciones.

Base de datos. Las credenciales viven en `db_config.ini`, que NO se sube al repositorio. La app las lee al conectarse; nunca están escritas en el código.

4 Estructura de carpetas

- `app.py`, `contenedor.py` — entrada y raíz de composición.
- `datos/` — conexión y repositorios (acceso a la base de datos).
- `servicios/` — lógica de negocio.
- `interfaz/` — pantallas y utilidades visuales.
- `pruebas/` — pruebas automáticas · `scripts/` — utilidades de datos.
- `recursos/logos/` — imágenes · `docs/` — documentación · `DB/` — respaldo.
- `maquetas_html/` — maquetas de las pantallas y del informe PDF (`10_informe_pdf.html`, con datos de ejemplo).

Las carpetas `datos/`, `servicios/` e `interfaz/` son **paquetes** de Python, por eso los módulos se importan cualificados (por ejemplo `import datos.repositorio` as `repositorio`).

5 Repositorio en GitHub

El código fuente de VorTrack está publicado en un repositorio público de GitHub:

Repositorio: <https://github.com/VorTrack-git/Vortrack.exe>

Para obtener el proyecto y ejecutarlo en otro equipo:

PASO	QUÉ HACER
1 · Clonar	<code>git clone https://github.com/VorTrack-git/Vortrack.exe.git</code> (o «Code → Download ZIP» desde la página).
2 · Dependencias	<code>python -m pip install customtkinter pyodbc pillow tkcalendar</code> y el controlador «ODBC Driver 18 for SQL Server».
3 · Base de datos	Restaurar <code>DB/vortrack_db.bacpac</code> en SQL Server y crear <code>db_config.ini</code> a partir de <code>db_config.example.ini</code> .
4 · Ejecutar	<code>python app.py</code> desde la carpeta raíz del proyecto.
5 · Probar	<code>python pruebas/pruebas_servicios.py</code> (sin base) y <code>python pruebas/pruebas_humo.py</code> (contra la base).

Importante. `db_config.ini` contiene las credenciales y está excluido del repositorio (`.gitignore`): cada equipo crea el suyo. Nunca se deben subir contraseñas a GitHub.

6 Carga en paralelo y experiencia de uso

Las pantallas que consultan la base de datos (Recolección, Transformación, Impresión e Histórico) ya no hacen sus consultas una por una: las lanzan **todas a la vez** y muestran un indicador de progreso mientras llegan los datos.

- **Consultas simultáneas.** Con `ThreadPoolExecutor` cada consulta corre en su propio hilo y con su propia conexión. Mientras una espera la respuesta del servidor, `pyodbc` libera el intérprete y las demás avanzan. En el Histórico, las 9 consultas pasaron de **6,4 s** en serie a **1,3 s** en paralelo.
- **La ventana no se congela.** La pantalla aparece al instante con un spinner. Tkinter no es seguro entre hilos, así que los hilos solo consultan datos; el hilo principal revisa cada 60 ms con `after()` si terminaron y recién ahí crea o rellena los widgets.
- **Spinner con porcentaje** (`interfaz/ui_cargando.py`). Un anillo que se llena según las consultas terminadas («2 de 3 consultas listas») con el mensaje de lo que se está cargando. Entre un paso y otro avanza despacio para no parecer congelado, pero solo llega al 100 % cuando todo terminó.
- **Cómo reutilizarlo.** `cargar_con_spinner(contenedor, {clave: (función, mensaje)}, al_terminar)` para varias consultas, o `ejecutar_con_spinner(contenedor, tarea, al_terminar)` para una tarea con etapas. Los formularios reciben los datos ya cargados mediante `precarga` y `resultado(...)`.
- **Menú «REGISTRAR» animado** (`interfaz/ui_desplegable.py`). Reemplaza al desplegable anterior, que mostraba «REGISTRAR» dos veces y con dos flechas. Ahora hay una sola flecha que gira hacia arriba al abrir, el panel se desliza y la página actual aparece resaltada.

7 Informe PDF

El botón **GENERAR INFORME** del Histórico produce un informe de gestión en PDF y lo guarda directamente en la carpeta **Descargas** del usuario.

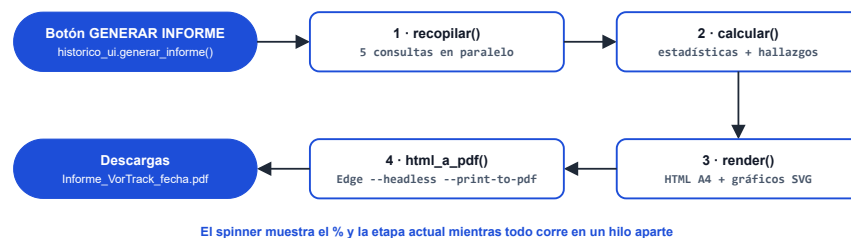


Fig. 3 – Generación del informe PDF (`servicios/servicio_informe.py`).

- **Contenido.** Portada, resumen ejecutivo (6 indicadores y 8 derivados), hallazgos clave redactados automáticamente, flujo del material, PET por mes y por lugar, ranking de estudiantes, piezas por estado y por modelo, producción de filamento, impresiones y detalle de jornadas.
- **Maqueta HTML → PDF.** El informe se diseña como HTML (`servicios/informe_html.py`) y Microsoft Edge en modo headless lo imprime a PDF. No hace falta instalar librerías de PDF y el resultado se ve igual que la maqueta. La plantilla con datos de ejemplo está en `maquetas_html/10_informe_pdf.html`.
- **Lógica separada y probada.** `ServicioInforme.calcular()` no toca la base ni la interfaz, por eso sus cálculos tienen pruebas en `pruebas_servicios.py`.
- **Control de calidad de los datos.** Si una producción registra más filamento usado del que produjo, el informe lo marca en rojo y lo avisa en los hallazgos.
- **Nombre del archivo.** `Informe_VorTrack_AAAA-MM-DD_HHMM.pdf`; si ya existe, se agrega «(2)», «(3)»...

8 Raíz y composición

app.py

134 líneas

Punto de entrada de la aplicación y orquestador de la navegación. Crea el **Contenedor** de dependencias y decide qué pantalla mostrar según el rol.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>import tkinter as tk</code>	Librería estándar de interfaz gráfica (ventanas y widgets).
<code>from tkinter import messagebox</code>	Librería estándar de interfaz gráfica (ventanas y widgets).
<code>import servicios.auth as auth</code>	Módulo propio · fachada de autenticación.
<code>import interfaz.inicio_ui as inicio_ui</code>	Módulo propio · pantalla de inicio.
<code>import interfaz.login_ui as login_ui</code>	Módulo propio · pantalla de login.
<code>import interfaz.recolec_ui as recolec_ui</code>	Módulo propio de VorTrack.
<code>import interfaz.transformacion_ui as transformacion_ui</code>	Módulo propio de VorTrack.
<code>import interfaz.impresion_ui as impresion_ui</code>	Módulo propio de VorTrack.
<code>import interfaz.historico_ui as historico_ui</code>	Módulo propio de VorTrack.
<code>import interfaz.admin_ui as admin_ui</code>	Módulo propio de VorTrack.
<code>from contenedor import Contenedor</code>	Módulo propio · raíz de composición.
<code>from interfaz.widgets import FabricaWidgets</code>	Módulo propio · fábrica de widgets (FabricaWidgets).
<code>import customtkinter as ctk</code>	Librería externa de widgets modernos (tema oscuro, esquinas redondeadas). La app funciona con o sin ella.

FUNCIONES Y CLASES PRINCIPALES

- `run()` — Bucle principal: muestra el login y luego alterna entre páginas hasta cerrar sesión o salir.
- `show_login()` — Abre la ventana de inicio de sesión; devuelve si el usuario entró.
- `show_main_app(page)` — Muestra la página pedida (inicio, recolección, admin, ...) inyectándole sus dependencias.
- `_create_root()` — Crea la ventana raíz (CTk si hay customtkinter, si no Tk).

contenedor.py

26 líneas

Raíz de composición: construye en un solo lugar las dependencias concretas (datos y servicios) para inyectarlas en la interfaz (principio DIP).

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>import datos.repositorio as repositorio</code>	Módulo propio · fachada de la capa de datos.
<code>from servicios.servicio_informe import ServicioInforme</code>	Módulo propio · servicio del informe PDF.
<code>from servicios.servicio_proyeccion import ServicioProyeccion</code>	Módulo propio · servicio de proyecciones.

FUNCIONES Y CLASES PRINCIPALES

- `class Contenedor` — Agrupa datos (repositorios), proyeccion (servicio de proyecciones) e informe (servicio del informe PDF) listos para repartir a cada pantalla.

9 Capa de datos (acceso a la base de datos)

datos/db.py

100 líneas

Conexión a SQL Server con pyodbc. Lee las credenciales de `db_config.ini` (fuera del repositorio) o de variables de entorno.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>import os</code>	Estándar: rutas y sistema de archivos.
<code>import configparser</code>	Estándar: leer el archivo <code>db_config.ini</code> .
<code>import pyodbc</code>	Librería externa para conectar con SQL Server mediante ODBC.

FUNCIONES Y CLASES PRINCIPALES

- `get_connection()` — Abre y devuelve una conexión a la base de datos.
- `test_connection()` — Prueba la conexión y devuelve (ok, mensaje).
- `_load_settings()` — Carga la configuración desde el `.ini` y el entorno.
- `_connection_string(cfg)` — Arma la cadena de conexión ODBC.

datos/repositorio_base.py

73 líneas

Base común de todos los repositorios. Recibe una *fábrica de conexión* inyectable, lo que facilita las pruebas.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>from datetime import date, datetime</code>	Estándar: manejo de fechas.
<code>import datos.db as db</code>	Módulo propio · conexión a la base de datos.

FUNCIONES Y CLASES PRINCIPALES

- `class RepositorioBase` — Ofrece helpers de acceso a datos a los repositorios hijos.
- `_query(sql, params, fetch)` — Ejecuta un SELECT y devuelve las filas.
- `_ejecutar(sql, params)` — Ejecuta INSERT/UPDATE/DELETE; puede devolver el id generado.
- `_valor(sql)` — Devuelve un único valor escalar.
- `parse_fecha(valor)` — Convierte texto a fecha admitiendo varios formatos.

datos/interfaces.py

77 líneas

Contratos (Protocols) de los repositorios. Las capas altas dependen de estos tipos abstractos, no de las clases concretas (DIP/ISP).

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>from typing import Any, Optional, Protocol, Sequence</code>	Estándar: anotaciones de tipos (Protocol, Optional).

FUNCIONES Y CLASES PRINCIPALES

- `Protocolos` — `RepoUsuariosProto`, `RepoJornadasProto`, `RepoProduccionProto`, `RepoFabricacionProto`, `RepoLugaresProto`, `RepoModelosProto`, `RepoResponsablesProto`, `RepoIndicadoresProto` — describen qué métodos debe ofrecer cada repositorio.

datos/repositorio.py

192 líneas

Fachada de la capa de datos: reúne bajo funciones simples las operaciones que la interfaz ya usaba y las delega a los repositorios por entidad.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>from datos.repositorio_base import parse_fecha</code>	Módulo propio · base común de los repositorios.
<code>from datos.repo_catalogos import RepoLugares, RepoModelos, RepoResponsables</code>	Módulo propio · repositorios de catálogos.
<code>from datos.repo_jornadas import RepoJornadas</code>	Módulo propio · repositorio de recolecciones.
<code>from datos.repo_produccion import RepoProduccion</code>	Módulo propio · repositorio de transformaciones.
<code>from datos.repo_fabricacion import RepoFabricacion</code>	Módulo propio · repositorio de impresiones.
<code>from datos.repo_usuarios import RepoUsuarios</code>	Módulo propio · repositorio de usuarios.
<code>from datos.repo_indicadores import RepoIndicadores</code>	Módulo propio · repositorio de indicadores.

FUNCIONES Y CLASES PRINCIPALES

- **Recolecciones** — `crear_jornada`, `recolecciones_recientes`, `recolecciones_admin`, `obtener/actualizar/eliminar_jornada`.
- **Transformaciones** — `crear_produccion`, `producciones_recientes/admin`, `obtener/actualizar/eliminar_produccion`.
- **Impresiones** — `crear_fabricacion`, `fabricaciones_recientes/admin`, `obtener/actualizar/eliminar_fabricacion`.
- **Catálogos** — `listar_responsables/lugares/modelos`, `crear/actualizar/eliminar_lugar` y `_modelo`.
- **Estudiantes e indicadores** — `listar/crear/actualizar/eliminar_estudiante`, `indicadores_resumen`, `ranking_participacion`.

datos/repo_usuarios.py

109 líneas

Repositorio de usuarios: consulta de login y CRUD de estudiantes. Un estudiante se crea como fila en Usuarios más su Responsable, enlazados por correo.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>from datos.repositorio_base import RepositorioBase</code>	Módulo propio · base común de los repositorios.

FUNCIONES Y CLASES PRINCIPALES

- **buscar_para_login(usuario)** — Busca la cuenta para validar el inicio de sesión.
- **listar_estudiantes()** — Lista estudiantes con su contraseña visible para el admin.
- **crear/actualizar/eliminar_estudiante(...)** — Alta, edición y baja de estudiantes (transacción).
- **_id_rol_estudiante(cur)** — Obtiene el id del rol Estudiante.

datos/repo_jornadas.py

74 líneas

Repositorio de recolecciones (tabla JornadasRecoleccion).

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>from datos.repositorio_base import RepositorioBase, parse_fecha</code>	Módulo propio · base común de los repositorios.

FUNCIONES Y CLASES PRINCIPALES

- **crear(...)** — Inserta una jornada y devuelve su id.
- **listar_para_combo() / peso_pet(id)** — Datos para los desplegables y el peso de PET de una jornada.
- **recientes() / listar_admin()** — Listados para las tablas.
- **obtener/actualizar/eliminar(id)** — Consulta, edición y borrado (con control de dependencias).

datos/repo_produccion.py

67 líneas

Repositorio de transformaciones (tabla ProduccionFilamento).

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>from datos.repositorio_base import RepositorioBase, parse_fecha</code>	Módulo propio · base común de los repositorios.

FUNCIONES Y CLASES PRINCIPALES

- `crear(...)` — Registra una producción de filamento.
- `listar_para_combo()` / `recientes()` / `listar_admin()` — Listados para combos y tablas.
- `obtener/actualizar/eliminar(id)` — Consulta, edición y borrado.

datos/repo_fabricacion.py

48 líneas

Repositorio de impresiones (tabla FabricacionObjetos).

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>from datos.repositorio_base import RepositorioBase, parse_fecha</code>	Módulo propio · base común de los repositorios.

FUNCIONES Y CLASES PRINCIPALES

- `crear(...)` — Registra una impresión/fabricación.
- `recientes()` / `listar_admin()` — Listados para tablas.
- `obtener/actualizar/eliminar(id)` — Consulta, edición y borrado.

datos/repo_catalogos.py

87 líneas

Repositorios de catálogos: Responsables, Lugares y Modelos (datos que alimentan los formularios).

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>from datos.repositorio_base import RepositorioBase, es_error_duplicado</code>	Módulo propio · base común de los repositorios.

FUNCIONES Y CLASES PRINCIPALES

- `RepoResponsables.listar()` — Estudiantes que pueden ser responsables.
- `RepoLugares` — `listar`, `listar_admin`, `crear`, `actualizar`, `eliminar` (verifica que no esté en uso).
- `RepoModelos` — `listar`, `listar_admin`, `crear`, `actualizar`, `eliminar` (con tiempo y peso estimados).

datos/repo_indicadores.py

54 líneas

Repositorio de indicadores agregados para el Histórico.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>from concurrent.futures import ThreadPoolExecutor</code>	Estándar: <code>ThreadPoolExecutor</code> para ejecutar varias consultas a la vez en hilos.
<code>from datos.repositorio_base import RepositorioBase</code>	Módulo propio · base común de los repositorios.

FUNCIONES Y CLASES PRINCIPALES

- `resumen()` — Devuelve los KPIs: PET, filamento, desperdicio, objetos, responsables, botellas. Lanza sus 6 consultas escalares **a la vez** con `ThreadPoolExecutor` (cada una en su propia conexión).
- `_SQL_KPIS` — Diccionario con las consultas de cada indicador.
- `ranking(limite)` — Devuelve el ranking de participación de los estudiantes.

10 Servicios (lógica de negocio)

servicios/auth.py

51 líneas

Fachada de autenticación: mantiene la API que usa el resto del programa y delega en ServicioAuth.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>from typing import Optional</code>	Estándar: anotaciones de tipos (Protocol, Optional).
<code>from servicios.servicio_auth import ServicioAuth, DatabaseUnavailable</code>	Módulo propio · servicio de autenticación.

FUNCIONES Y CLASES PRINCIPALES

- `authenticate(u, p)` — Valida credenciales; lanza `DatabaseUnavailable` si falla la BD.
- `is_admin()` / `is_authenticated()` / `get_current_user()` — Estado de la sesión.
- `require_auth()` — Exige sesión activa.
- `save/clear/load_remembered_user()` — Gestiona el 'recordar usuario'.

servicios/servicio_auth.py

91 líneas

Servicio de autenticación: login, sesión y roles. Depende de un repositorio de usuarios (abstracción) inyectado.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>import hashlib</code>	Estándar: cifrar contraseñas con SHA-256.
<code>from typing import Optional</code>	Estándar: anotaciones de tipos (Protocol, Optional).
<code>from datos.repo_usuarios import RepoUsuarios</code>	Módulo propio · repositorio de usuarios.
<code>from servicios.almacenamiento_sesion import AlmacenamientoSesion</code>	Módulo propio · 'recordar usuario'.

FUNCIONES Y CLASES PRINCIPALES

- `class ServicioAuth` — Coordina login, sesión y política de contraseñas.
- `autenticar(u, p)` — Verifica el usuario y abre la sesión.
- `_hash(texto)` / `_password_matches(...)` — SHA-256 para el admin; compara texto plano o hash.
- `es_admin()` / `requerir_auth()` — Rol y control de acceso.
- `class DatabaseUnavailable` — Error que indica que la base no respondió.

servicios/servicio_proyeccion.py

56 líneas

Servicio de proyecciones de producción. Centraliza los cálculos que antes estaban en la interfaz.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
-------------	----------------

FUNCIONES Y CLASES PRINCIPALES

- `proyeccion_impresion(tiempo_h, peso_g, cantidad)` — Devuelve el peso total (g) y el tiempo total (min) de una impresión.
- `piezas_por_filamento(peso_kg, modelos)` — Calcula cuántas piezas de cada modelo alcanzan y el sobrante.

servicios/almacenamiento_sesion.py

43 líneas

Guarda localmente el correo 'recordado' para autocompletar el login. Se separa de la autenticación (SRP).

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>import json</code>	Estándar: leer y escribir archivos JSON.
<code>import os</code>	Estándar: rutas y sistema de archivos.

FUNCIONES Y CLASES PRINCIPALES

- **class AlmacenamientoSesion** — guardar(usuario), cargar(), limpiar() sobre un archivo JSON local.

servicios/validaciones.py

54 líneas

Validadores reutilizables para los formularios (evitan repetir reglas en cada pantalla).

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>from datetime import datetime</code>	Estándar: manejo de fechas.

FUNCIONES Y CLASES PRINCIPALES

- **es_vacio(texto)** — ¿El campo está vacío?
- **numero / numero_positivo / entero_positivo** — Validan y convierten valores numéricos.
- **fecha_valida(texto)** — Acepta dd/mm/aaaa y otros formatos.
- **correo_valido(texto)** — Comprueba un correo básico.

servicios/servicio_informe.py

239 líneas

Servicio del informe PDF. Orquesta todo el proceso: recopila los datos, calcula las estadísticas, arma el HTML y lo exporta a PDF en la carpeta Descargas.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>from collections import OrderedDict, defaultdict</code>	Estándar: OrderedDict y defaultdict para agrupar estadísticas.
<code>from concurrent.futures import ThreadPoolExecutor</code>	Estándar: ThreadPoolExecutor para ejecutar varias consultas a la vez en hilos.
<code>from datetime import date, datetime</code>	Estándar: manejo de fechas.
<code>import servicios.exportador_pdf as exportador_pdf</code>	Módulo propio · conversión HTML → PDF.
<code>from servicios import informe_html</code>	Módulo propio · maqueta HTML del informe.
<code>from servicios.informe_html import num as n</code>	Módulo propio · maqueta HTML del informe.

FUNCIONES Y CLASES PRINCIPALES

- **class ServicioInforme** — Recibe la capa de datos y el exportador (inyectables).
- **recopilar()** — Lanza a la vez las 5 consultas del informe (indicadores, ranking, recolecciones, producciones, impresiones).
- **calcular(crudos)** — Lógica pura: rendimiento, desperdicio, PET pendiente, filamento usado/disponible, tasa de éxito, PET por mes y por lugar, piezas por modelo y estado.
- **hallazgos(modelo)** — Redacta automáticamente las frases de «Hallazgos clave», incluidos avisos de datos inconsistentes.
- **generar(usuario, progreso, carpeta)** — Ejecuta las 4 etapas avisando el avance con progreso(paso, total, texto) y devuelve la ruta del PDF.

servicios/informe_html.py

436 líneas

Maqueta HTML del informe (A4, lista para imprimir). Documento autocontenido: estilos en línea, logo en base64 y gráficos dibujados en SVG, sin depender de internet.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>import base64</code>	Estándar: incrustar el logo dentro del HTML del informe.
<code>import html</code>	Estándar: escapar texto de usuario al generar HTML.
<code>from pathlib import Path</code>	Estándar: rutas de archivos orientadas a objetos.
<code>import math</code>	Estándar: trigonometría para dibujar y rotar la flecha y la dona.

FUNCIONES Y CLASES PRINCIPALES

- `render(modelo, generado, usuario, logo_uri)` — Devuelve el HTML completo: portada, resumen, hallazgos, gráficos y tablas.
- `svg_barras_h` / `svg_columnas` / `svg_dona` / `html_embudo` — Dibujan los gráficos (barras, columnas por mes, dona de estados y flujo del material).
- `_tabla(columnas, filas, pie)` — Tabla con encabezado, filas alternadas y fila de totales.
- `num(x, dec)` / `fecha(d)` / `minutos(m)` — Formato español de números (1.234,56), fechas y duraciones.
- `logo_data_uri()` — Incrusta el logo de VorTrack en el documento.

servicios/exportador_pdf.py

122 líneas

Convierte HTML a PDF con el navegador del sistema en modo headless (Microsoft Edge, o Chrome si no hay Edge). No requiere instalar librerías de PDF.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>import os</code>	Estándar: rutas y sistema de archivos.
<code>import shutil</code>	Estándar: buscar el navegador en el PATH y mover el PDF final.
<code>import subprocess</code>	Estándar: ejecutar Microsoft Edge/Chrome en modo headless.
<code>import sys</code>	Estándar: acceso al intérprete (ajustar la ruta de módulos).
<code>import tempfile</code>	Estándar: carpeta temporal para el HTML y el perfil del navegador.
<code>from pathlib import Path</code>	Estándar: rutas de archivos orientadas a objetos.
<code>import ctypes</code>	Estándar: llamar a la API de Windows para ubicar la carpeta Descargas.
<code>from ctypes import wintypes</code>	Estándar: llamar a la API de Windows para ubicar la carpeta Descargas.
<code>import uuid</code>	Estándar: identificador (GUID) de la carpeta Descargas de Windows.

FUNCIONES Y CLASES PRINCIPALES

- `buscar_navegador()` — Ubica msedge.exe / chrome.exe en las rutas habituales o en el PATH.
- `html_a_pdf(html, destino)` — Escribe el HTML temporal y ejecuta `--headless --print-to-pdf` con un perfil propio.
- `carpeta_descargas()` — Obtiene la carpeta Descargas real del usuario (API de Windows), aunque se haya movido.
- `ruta_disponible(carpeta, nombre)` — Evita sobrescribir: agrega «(2)», «(3)»... si el archivo ya existe.
- `class ErrorExportacion` — Error con un mensaje claro cuando no se pudo generar el PDF.

11 Interfaz de usuario

interfaz/ui_tema.py

31 líneas

Paleta de colores (COLORS) y tipografía (FONT_FAMILY) compartidas por toda la interfaz.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
-------------	----------------

FUNCIONES Y CLASES PRINCIPALES

- **COLORS** — Diccionario con los colores de la marca (fondo, superficies, cian, etc.).
- **FONT_FAMILY** — Fuente base de la interfaz.

interfaz/ui_imagenes.py

37 líneas

Carga de imágenes según la librería disponible y resolución de rutas a recursos/logos/.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>import os</code>	Estándar: rutas y sistema de archivos.
<code>from PIL import Image, ImageTk</code>	Pillow (externa): cargar y redimensionar imágenes (logo e íconos).
<code>import customtkinter as ctk</code>	Librería externa de widgets modernos (tema oscuro, esquinas redondeadas). La app funciona con o sin ella.

FUNCIONES Y CLASES PRINCIPALES

- **load_image(path, size)** — Devuelve la imagen como CTkImage o ImageTk.
- **ruta_logo(nombre)** — Ruta absoluta a una imagen de recursos/logos/.

interfaz/ui_ventana.py

80 líneas

Utilidades de ventana.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>import os</code>	Estándar: rutas y sistema de archivos.
<code>import tkinter as tk</code>	Librería estándar de interfaz gráfica (ventanas y widgets).
<code>from PIL import Image, ImageTk</code>	Pillow (externa): cargar y redimensionar imágenes (logo e íconos).

FUNCIONES Y CLASES PRINCIPALES

- **maximize_window(root)** — Abre la ventana maximizada (con reintento para customtkinter).
- **set_window_icon(root)** — Fija el ícono de la ventana.

interfaz/ui_dialogos.py

135 líneas

Diálogos modales, formularios con scroll y enlaces del pie de página.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>import tkinter as tk</code>	Librería estándar de interfaz gráfica (ventanas y widgets).
<code>import customtkinter as ctk</code>	Librería externa de widgets modernos (tema oscuro, esquinas redondeadas). La app funciona con o sin ella.
<code>import interfaz.soporte_dialog as soporte_dialog</code>	Módulo propio · diálogo de soporte.
<code>import interfaz.privacidad as privacidad</code>	Módulo propio · diálogo de privacidad.
<code>import interfaz.use_terms as use_terms</code>	Módulo propio · diálogo de términos.

FUNCIONES Y CLASES PRINCIPALES

- `scrollable_form(parent, bg)` — Contenedor con scroll vertical para formularios largos.
- `text_dialog(...)` — Ventana con texto largo (privacidad/términos).
- `open_footer_link` / `bind_footer_link` — Hacen clicables los enlaces del pie.

interfaz/ui_navbar.py

125 líneas

Barra de navegación y pie de página estándar de las pantallas autenticadas.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>import tkinter as tk</code>	Librería estándar de interfaz gráfica (ventanas y widgets).
<code>from interfaz.ui_tema import COLORS, FONT_FAMILY</code>	Módulo propio · paleta y tipografía.
<code>from interfaz.ui_dialogos import bind_footer_link</code>	Módulo propio · diálogos y scroll.
<code>from interfaz.ui_desplegable import MenuDesplegable</code>	Módulo propio · menú desplegable animado.
<code>import customtkinter as ctk</code>	Librería externa de widgets modernos (tema oscuro, esquinas redondeadas). La app funciona con o sin ella.
<code>import servicios.auth as auth</code>	Módulo propio · fachada de autenticación.

FUNCIONES Y CLASES PRINCIPALES

- `build_app_navbar(app)` — Construye la barra superior (Registrar, Histórico, Admin, Logout).
- `build_registrar_menu(parent, app)` — Crea el menú animado «REGISTRAR» (`MenuDesplegable`) y resalta la página actual según `app.PAGE`. También lo usa la pantalla de inicio.
- `build_app_footer(app)` — Construye el pie con marca y enlaces.

interfaz/ui_desplegable.py

206 líneas

Menú desplegable animado de la barra de navegación («REGISTRAR»). Funciona igual con o sin `customtkinter`.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>import math</code>	Estándar: trigonometría para dibujar y rotar la flecha y la dona.
<code>import tkinter as tk</code>	Librería estándar de interfaz gráfica (ventanas y widgets).
<code>from interfaz.ui_tema import COLORS, FONT_FAMILY</code>	Módulo propio · paleta y tipografía.

FUNCIONES Y CLASES PRINCIPALES

- `class MenuDesplegable` — Etiqueta con flecha que gira hacia arriba al abrir y panel de opciones que se desliza.
- `abrir()` / `cerrar()` / `alternar()` — Controlan el menú con animación suave (~200 ms).
- `_clic_fuera(evento)` — Cierra el menú al hacer clic fuera de él; también se cierra con Escape.

interfaz/ui_cargando.py

212 líneas

Indicador de carga reutilizable: un anillo con porcentaje dibujado en Canvas, más los ayudantes que ejecutan trabajo en hilos sin congelar la ventana.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>import time</code>	Estándar: medir tiempos (duración mínima del spinner).
<code>import tkinter as tk</code>	Librería estándar de interfaz gráfica (ventanas y widgets).
<code>from concurrent.futures import ThreadPoolExecutor</code>	Estándar: <code>ThreadPoolExecutor</code> para ejecutar varias consultas a la vez en hilos.
<code>from interfaz.ui_tema import COLORS, FONT_FAMILY</code>	Módulo propio · paleta y tipografía.

FUNCIONES Y CLASES PRINCIPALES

- **class SpinnerCarga** — Anillo de progreso con % en el centro, mensaje y detalle; tapa el contenedor mientras carga.
- **cargar_con_spinner(contenedor, cargas, al_terminar)** — Lanza varias consultas a la vez; el % avanza según cuántas terminaron. Al final llama `al_terminar(futuros)`.
- **ejecutar_con_spinner(contenedor, tarea, al_terminar)** — Corre una tarea larga que reporta sus etapas (se usa para el informe PDF).
- **resultado(precarga, clave, funcion)** — Devuelve el dato precargado o, si no hay, lo consulta en el momento.

interfaz/ui_utils.py

44 líneas

Fachada de utilidades de interfaz: reexporta todo lo anterior para que el código use `ui_utils.*` sin cambios.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>from interfaz.ui_tema import COLORS, FONT_FAMILY</code>	Módulo propio · paleta y tipografía.
<code>from interfaz.ui_imagenes import load_image, ruta_logo, DIR_LOGOS</code>	Módulo propio · carga de imágenes.
<code>from interfaz.ui_ventana import maximize_window, set_window_icon</code>	Módulo propio · maximizar ventana e ícono.
<code>from interfaz.ui_dialogos import scrollable_form, text_dialog, open_footer_link, bind_footer_link</code>	Módulo propio · diálogos y scroll.
<code>from interfaz.ui_navbar import build_app_navbar, build_app_footer</code>	Módulo propio · barra de navegación y pie.
<code>from interfaz.widgets import FabricaWidgets</code>	Módulo propio · fábrica de widgets (<code>FabricaWidgets</code>).

FUNCIONES Y CLASES PRINCIPALES

- **Reexporta** — `COLORS`, `FONT_FAMILY`, `load_image`, `ruta_logo`, `maximize_window`, `set_window_icon`, `scrollable_form`, `text_dialog`, `build_app_navbar`, `build_app_footer`, `FabricaWidgets`.

interfaz/widgets.py

209 líneas

Fábrica de widgets: centraliza en un solo lugar la creación de controles para las dos ramas (customtkinter y tkinter puro).

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>import tkinter as tk</code>	Librería estándar de interfaz gráfica (ventanas y widgets).
<code>from tkinter import ttk</code>	Librería estándar de interfaz gráfica (ventanas y widgets).
<code>from interfaz.ui_tema import COLORS, FONT_FAMILY</code>	Módulo propio · paleta y tipografía.
<code>import customtkinter as ctk</code>	Librería externa de widgets modernos (tema oscuro, esquinas redondeadas). La app funciona con o sin ella.

FUNCIONES Y CLASES PRINCIPALES

- `class FabricaWidgets` — Crea de forma uniforme: `label_campo`, `titulo`, `hint`, `entry`, `combo`, `textbox`, `boton`, `tarjeta` y `tabla`.
- `tabla(parent, columnas, estilo)` — Crea una tabla (Treeview) estilizada.
- `estilo_tabla(nombre)` — Configura el estilo oscuro de las tablas.

interfaz/login_ui.py

419 líneas

Pantalla de inicio de sesión. Recibe el servicio de autenticación y valida las credenciales.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>import os</code>	Estándar: rutas y sistema de archivos.
<code>import tkinter as tk</code>	Librería estándar de interfaz gráfica (ventanas y widgets).
<code>from tkinter import messagebox</code>	Librería estándar de interfaz gráfica (ventanas y widgets).
<code>import servicios.auth as auth</code>	Módulo propio · fachada de autenticación.
<code>import interfaz.ui_utils as ui_utils</code>	Módulo propio · fachada de utilidades de interfaz.
<code>import servicios.validaciones as validaciones</code>	Módulo propio · validadores de formularios.
<code>import customtkinter as ctk</code>	Librería externa de widgets modernos (tema oscuro, esquinas redondeadas). La app funciona con o sin ella.
<code>import interfaz.soporte_dialog as soporte_dialog</code>	Módulo propio · diálogo de soporte.
<code>from app import run</code>	Módulo propio · punto de entrada.
<code>import interfaz.inicio_ui as inicio_ui</code>	Módulo propio · pantalla de inicio.

FUNCIONES Y CLASES PRINCIPALES

- `class LoginApp` — Construye la tarjeta de login.
- `login()` — Valida y entra; muestra errores de credenciales o de conexión.
- `_build_form(frame)` — Arma los campos de usuario y contraseña.
- `_toggle_password()` / `_load_remembered_user()` — Ver/ocultar contraseña y precargar el correo.

Pantalla de inicio ('Quiénes somos'). Punto de partida tras entrar.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>import os</code>	Estándar: rutas y sistema de archivos.
<code>import sys</code>	Estándar: acceso al intérprete (ajustar la ruta de módulos).
<code>import tkinter as tk</code>	Librería estándar de interfaz gráfica (ventanas y widgets).
<code>from tkinter import messagebox</code>	Librería estándar de interfaz gráfica (ventanas y widgets).
<code>import servicios.auth as auth</code>	Módulo propio · fachada de autenticación.
<code>import interfaz.ui_utils as ui_utils</code>	Módulo propio · fachada de utilidades de interfaz.
<code>from interfaz.ui_navbar import build_registrar_menu</code>	Módulo propio · barra de navegación y pie.
<code>from PIL import Image, ImageTk</code>	Pillow (externa): cargar y redimensionar imágenes (logo e íconos).
<code>import customtkinter as ctk</code>	Librería externa de widgets modernos (tema oscuro, esquinas redondeadas). La app funciona con o sin ella.
<code>from app import run</code>	Módulo propio · punto de entrada.
<code>import interfaz.login_ui as login_ui</code>	Módulo propio · pantalla de login.

FUNCIONES Y CLASES PRINCIPALES

- `class VorTrackApp` — Arma navbar, contenido de bienvenida y pie.
- `explorar_sistema()` / `on_registrar_select()` / `ir_a_historico()` — Navegación hacia las demás pantallas.
- `build_navbar()` — Barra propia con la pestaña activa subrayada; el menú «REGISTRAR» se crea con `build_registrar_menu`.

Pantalla para registrar recolecciones de PET. Recibe la capa de datos y la fábrica de widgets.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>import os</code>	Estándar: rutas y sistema de archivos.
<code>import tkinter as tk</code>	Librería estándar de interfaz gráfica (ventanas y widgets).
<code>from tkinter import messagebox, ttk</code>	Librería estándar de interfaz gráfica (ventanas y widgets).
<code>from datetime import datetime</code>	Estándar: manejo de fechas.
<code>import servicios.auth as auth</code>	Módulo propio · fachada de autenticación.
<code>import interfaz.ui_utils as ui_utils</code>	Módulo propio · fachada de utilidades de interfaz.
<code>import datos.repositorio as repositorio</code>	Módulo propio · fachada de la capa de datos.
<code>import servicios.validaciones as validaciones</code>	Módulo propio · validadores de formularios.
<code>from interfaz.ui_tema import COLORS, FONT_FAMILY</code>	Módulo propio · paleta y tipografía.
<code>from interfaz.ui_cargando import cargar_con_spinner, resultado</code>	Módulo propio · spinner con porcentaje y carga en paralelo.
<code>from interfaz.widgets import FabricaWidgets</code>	Módulo propio · fábrica de widgets (FabricaWidgets).
<code>from tkcalendar import DateEntry</code>	Librería externa: selector de fecha (DateEntry).
<code>import customtkinter as ctk</code>	Librería externa de widgets modernos (tema oscuro, esquinas redondeadas). La app funciona con o sin ella.
<code>import interfaz.login_ui as login_ui</code>	Módulo propio · pantalla de login.

FUNCIONES Y CLASES PRINCIPALES

- **class `RecoleccionForm`** — Formulario de nueva recolección. Recibe precarga con los catálogos ya consultados.
- **`registrar()`** — Valida y guarda la jornada.
- **class `RecentRecordsFrame`** — Tabla de últimos registros; recargar(precarga) la actualiza.
- **class `RecoleccionApp`** — Ensambla la pantalla: lanza en paralelo responsables, lugares y recientes con `cargar_con_spinner` y arma los paneles en `_construir_paneles(cargas)`.

Pantalla para registrar la transformación en filamento. Calcula el desperdicio automáticamente.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>import os</code>	Estándar: rutas y sistema de archivos.
<code>import tkinter as tk</code>	Librería estándar de interfaz gráfica (ventanas y widgets).
<code>from tkinter import messagebox, ttk</code>	Librería estándar de interfaz gráfica (ventanas y widgets).
<code>from datetime import datetime</code>	Estándar: manejo de fechas.
<code>import servicios.auth as auth</code>	Módulo propio · fachada de autenticación.
<code>import interfaz.ui_utils as ui_utils</code>	Módulo propio · fachada de utilidades de interfaz.
<code>import datos.repositorio as repositorio</code>	Módulo propio · fachada de la capa de datos.
<code>import servicios.validaciones as validaciones</code>	Módulo propio · validadores de formularios.
<code>from interfaz.ui_tema import COLORS, FONT_FAMILY</code>	Módulo propio · paleta y tipografía.
<code>from interfaz.ui_cargando import cargar_con_spinner, resultado</code>	Módulo propio · spinner con porcentaje y carga en paralelo.
<code>from interfaz.widgets import FabricaWidgets</code>	Módulo propio · fábrica de widgets (FabricaWidgets).
<code>from tkcalendar import DateEntry</code>	Librería externa: selector de fecha (DateEntry).
<code>import customtkinter as ctk</code>	Librería externa de widgets modernos (tema oscuro, esquinas redondeadas). La app funciona con o sin ella.
<code>import interfaz.login_ui as login_ui</code>	Módulo propio · pantalla de login.

FUNCIONES Y CLASES PRINCIPALES

- `class TransformacionForm` — Formulario de producción de filamento.
- `_on_jornada()` / `_update_desperdicio()` — Autocompletan el peso ingresado y calculan el desperdicio.
- `registrar()` — Valida y guarda la producción.
- `class RecentTransformacionesFrame / TransformacionApp` — Historial y ensamblado. Jornadas e historial se cargan en paralelo con `spinner(_construir_paneles)`.

Pantalla para registrar impresiones 3D. Proyecta peso y tiempo según el modelo.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>import os</code>	Estándar: rutas y sistema de archivos.
<code>import tkinter as tk</code>	Librería estándar de interfaz gráfica (ventanas y widgets).
<code>from tkinter import messagebox, ttk</code>	Librería estándar de interfaz gráfica (ventanas y widgets).
<code>from datetime import datetime</code>	Estándar: manejo de fechas.
<code>import servicios.auth as auth</code>	Módulo propio · fachada de autenticación.
<code>import interfaz.ui_utils as ui_utils</code>	Módulo propio · fachada de utilidades de interfaz.
<code>import datos.repositorio as repositorio</code>	Módulo propio · fachada de la capa de datos.
<code>import servicios.validaciones as validaciones</code>	Módulo propio · validadores de formularios.
<code>from interfaz.ui_tema import COLORS, FONT_FAMILY</code>	Módulo propio · paleta y tipografía.
<code>from interfaz.ui_cargando import cargar_con_spinner, resultado</code>	Módulo propio · spinner con porcentaje y carga en paralelo.
<code>from interfaz.widgets import FabricaWidgets</code>	Módulo propio · fábrica de widgets (FabricaWidgets).
<code>from servicios.servicio_proyeccion import ServicioProyeccion</code>	Módulo propio · servicio de proyecciones.
<code>from tkcalendar import DateEntry</code>	Librería externa: selector de fecha (DateEntry).
<code>import customtkinter as ctk</code>	Librería externa de widgets modernos (tema oscuro, esquinas redondeadas). La app funciona con o sin ella.
<code>import interfaz.login_ui as login_ui</code>	Módulo propio · pantalla de login.

FUNCIONES Y CLASES PRINCIPALES

- `class ImpresionForm` — Formulario de impresión.
- `_proyectar()` — Autocompleta peso (g) y tiempo (min) con el servicio de proyección.
- `registrar()` — Valida y guarda la fabricación.
- `class RecentImpresionesFrame / ImpresionApp` — Historial y ensamblado. Modelos, filamentos e historial se cargan en paralelo con spinner (`_construir_paneles`).

Tablero de histórico: indicadores, ranking y proyecciones de producción.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>import os</code>	Estándar: rutas y sistema de archivos.
<code>import tkinter as tk</code>	Librería estándar de interfaz gráfica (ventanas y widgets).
<code>from tkinter import messagebox, ttk</code>	Librería estándar de interfaz gráfica (ventanas y widgets).
<code>import servicios.auth as auth</code>	Módulo propio · fachada de autenticación.
<code>import interfaz.ui_utils as ui_utils</code>	Módulo propio · fachada de utilidades de interfaz.
<code>import datos.repositorio as repositorio</code>	Módulo propio · fachada de la capa de datos.
<code>from interfaz.ui_tema import COLORS, FONT_FAMILY</code>	Módulo propio · paleta y tipografía.
<code>from interfaz.ui_cargando import cargar_con_spinner, ejecutar_con_spinner</code>	Módulo propio · spinner con porcentaje y carga en paralelo.
<code>from interfaz.widgets import FabricaWidgets</code>	Módulo propio · fábrica de widgets (FabricaWidgets).
<code>from servicios.servicio_informe import ServicioInforme</code>	Módulo propio · servicio del informe PDF.
<code>from servicios.servicio_proyeccion import ServicioProyeccion</code>	Módulo propio · servicio de proyecciones.
<code>import customtkinter as ctk</code>	Librería externa de widgets modernos (tema oscuro, esquinas redondeadas). La app funciona con o sin ella.
<code>import interfaz.login_ui as login_ui</code>	Módulo propio · pantalla de login.

FUNCIONES Y CLASES PRINCIPALES

- `class HistoricoApp` — Arma KPIs, ranking y el panel de proyecciones. Recibe el servicio informe inyectado.
- `_kpi_card()` / `_build_ranking()` / `_build_proyecciones()` — Construyen el esqueleto de cada bloque; se muestran con «—» mientras cargan.
- `_poblar(cargas)` — Cuando terminan las 4 consultas en paralelo, rellena indicadores, ranking y proyecciones.
- `_proy_calcular()` — Calcula, con el servicio, cuántas piezas alcanzan por filamento.
- `generar_informe()` / `_informe_listo(futuro)` — Genera el informe PDF en un hilo con spinner por etapas, lo guarda en Descargas y ofrece abrirlo.

interfaz/admin_ui.py

1001 líneas

Panel de administración (solo administradores). Seis secciones CRUD con tablas ordenables. Conserva sus propios helpers de widgets.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>import os</code>	Estándar: rutas y sistema de archivos.
<code>import tkinter as tk</code>	Librería estándar de interfaz gráfica (ventanas y widgets).
<code>from tkinter import messagebox, ttk</code>	Librería estándar de interfaz gráfica (ventanas y widgets).
<code>import servicios.auth as auth</code>	Módulo propio · fachada de autenticación.
<code>import interfaz.ui_utils as ui_utils</code>	Módulo propio · fachada de utilidades de interfaz.
<code>import datos.repositorio as repositorio</code>	Módulo propio · fachada de la capa de datos.
<code>import servicios.validaciones as validaciones</code>	Módulo propio · validadores de formularios.
<code>from interfaz.ui_tema import COLORS, FONT_FAMILY</code>	Módulo propio · paleta y tipografía.
<code>import customtkinter as ctk</code>	Librería externa de widgets modernos (tema oscuro, esquinas redondeadas). La app funciona con o sin ella.
<code>import interfaz.login_ui as login_ui</code>	Módulo propio · pantalla de login.

FUNCIONES Y CLASES PRINCIPALES

- `class AdminApp` — Ventana con selector de sección y contenido dinámico.
- `_show_estudiantes` / `_show_lugares` / `_show_modelos` — Vistas de catálogos y cuentas.
- `_show_recolecciones` / `_show_transformaciones` / `_show_impresiones` — Edición de registros de producción.
- `_est_*` / `_lug_*` / `_mod_*` / `_rec_*` / `_trans_*` / `_imp_*` — Grupos de métodos recargar/leer/guardar/editar/eliminar por sección.
- `_tree(...)` / `_sort_tree(...)` — Crean y ordenan las tablas.

interfaz/soporte_dialog.py

137 líneas

Ventana de Soporte Técnico compartida.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>import tkinter as tk</code>	Librería estándar de interfaz gráfica (ventanas y widgets).

FUNCIONES Y CLASES PRINCIPALES

- `abrir_soporte(root)` — Muestra el diálogo con los datos de contacto.

interfaz/privacidad.py

43 líneas

Diálogo de Política de Privacidad.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>import interfaz.ui_utils as ui_utils</code>	Módulo propio · fachada de utilidades de interfaz.

FUNCIONES Y CLASES PRINCIPALES

- `abrir_privacidad(root)` — Muestra el texto de privacidad.

interfaz/use_terms.py

45 líneas

Diálogo de Términos de Uso.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>import interfaz.ui_utils as ui_utils</code>	Módulo propio · fachada de utilidades de interfaz.

FUNCIONES Y CLASES PRINCIPALES

- `abrir_terminos(root)` — Muestra el texto de términos.

12 Pruebas

pruebas/pruebas_humo.py

104 líneas

Prueba de humo: ejercita el CRUD de cada área contra la base real con datos temporales que borra al final.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>import os as _os</code>	Estándar: rutas y sistema de archivos.
<code>import sys as _sys</code>	Estándar: acceso al intérprete (ajustar la ruta de módulos).
<code>import datos.repositorio as R</code>	Módulo propio · fachada de la capa de datos.
<code>import servicios.auth as auth</code>	Módulo propio · fachada de autenticación.

FUNCIONES Y CLASES PRINCIPALES

- `main()` — Corre las 16 verificaciones.
- `check(cond, msg)` — Registra el resultado de cada verificación.

pruebas/pruebas_servicios.py

107 líneas

Pruebas unitarias de servicios y validaciones (sin base de datos).

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>import os as _os</code>	Estándar: rutas y sistema de archivos.
<code>import sys as _sys</code>	Estándar: acceso al intérprete (ajustar la ruta de módulos).
<code>from datetime import date, datetime</code>	Estándar: manejo de fechas.
<code>from servicios import informe_html</code>	Módulo propio · maqueta HTML del informe.
<code>from servicios.servicio_informe import ServicioInforme</code>	Módulo propio · servicio del informe PDF.
<code>from servicios.servicio_proyeccion import ServicioProyeccion</code>	Módulo propio · servicio de proyecciones.
<code>import servicios.validaciones as v</code>	Módulo propio · validadores de formularios.

FUNCIONES Y CLASES PRINCIPALES

- `main()` — Corre las 34 comprobaciones: proyecciones, validadores y estadísticas del informe PDF (rendimiento, desperdicio, PET por mes, alertas de datos, formato de números).

scripts/seed_datos.py

73 líneas

Siembra los catálogos base (Lugares, Responsables, Modelos), necesarios como llaves foráneas.

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>import os as _os</code>	Estándar: rutas y sistema de archivos.
<code>import sys as _sys</code>	Estándar: acceso al intérprete (ajustar la ruta de módulos).
<code>import datos.db as db</code>	Módulo propio · conexión a la base de datos.

FUNCIONES Y CLASES PRINCIPALES

- `main()` — Inserta los catálogos si no existen (idempotente).

scripts/seed_usuario.py

87 líneas

Crea o actualiza el primer usuario administrador (rol + cuenta con contraseña por consola).

IMPORTACIONES

IMPORTACIÓN	PARA QUÉ SIRVE
<code>import getpass</code>	Estándar: leer una contraseña por consola sin mostrarla.
<code>import hashlib</code>	Estándar: cifrar contraseñas con SHA-256.
<code>import os as _os</code>	Estándar: rutas y sistema de archivos.
<code>import sys as _sys</code>	Estándar: acceso al intérprete (ajustar la ruta de módulos).
<code>import datos.db as db</code>	Módulo propio · conexión a la base de datos.

FUNCIONES Y CLASES PRINCIPALES

- `main()` — Asegura el rol Administrador y crea/actualiza el usuario.
- `_hash(pwd)` — SHA-256 de la contraseña.
- `_get_or_create_rol(cur)` — Garantiza el rol Administrador.